

Lecture 11 :

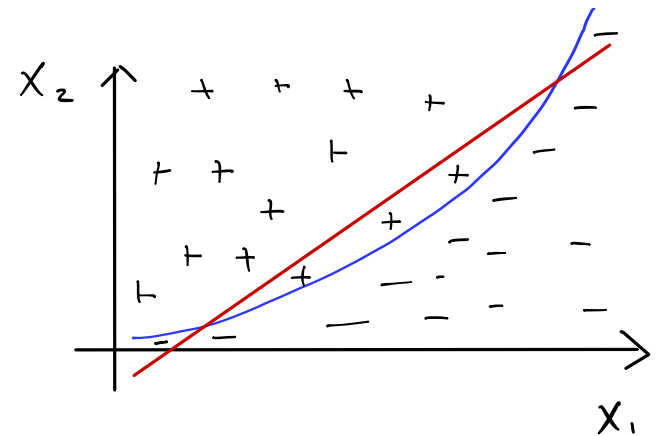
Kernel Methods

Kernel Regression

so far: $\hat{y} = w_0 + w_1 x_1 + w_2 x_2 = \underline{w}^T \underline{x} \quad (p=3)$

new: $\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_1^2 + w_4 x_1 x_2 + w_5 x_2^2 \quad (d=6)$

$$\text{let } \phi(\underline{x}) = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_1^2 \\ x_1 x_2 \\ x_2^2 \end{bmatrix} \Rightarrow \hat{y} = \underline{w}^T \underbrace{\phi(\underline{x})}_{\phi(\underline{x}) \in \mathbb{R}^6}$$



Kernel methods formalize and generalize this idea

original feature vector $\underline{x} \in \mathbb{R}^p \xrightarrow{\text{map to}} \phi(\underline{x}) \in \mathbb{R}^d$ "high-dimensional feature space"
typically $d > p$

goal: find weights s.t. $\hat{y} = \hat{\underline{w}}^T \phi(\underline{x})$ gives good predictions

Kernel methods allow us to do this without computing $\phi(x_i)$
explicitly for all training samples $\underline{x}_i$:
this is especially helpful because in many cases $d \rightarrow \infty$

for degree 5 polynomials:

$$\phi(\underline{x}) = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_p \\ x_1 x_2 \\ x_1 x_3 \\ \vdots \\ x_1 x_p \\ x_2 x_3 \\ \vdots \\ x_p^2 \\ x_1 x_2 x_3 \\ x_1 x_2 x_4 \\ \vdots \\ x_1 x_2 x_3 x_4 \\ x_1 x_2 x_3 x_5 \\ \vdots \\ x_1 x_2 x_3 x_4 x_5 \\ x_1 x_2 x_3 x_4 x_6 \\ \vdots \end{bmatrix}$$

How do we do this?

$$d \approx p^5$$

Recall Ridge Regression

$$\min_{\underline{w}} \underbrace{\|y - X\underline{w}\|_2^2}_{f(\underline{w})} + \lambda \|\underline{w}\|_2^2. \quad \text{Let } X = U\Sigma V^T.$$

Compute gradient and set to zero:

$$\begin{aligned} \Rightarrow \hat{\underline{w}} &= (X^T X + \lambda I)^{-1} X^T y \\ &= V (\underbrace{\Sigma^T \Sigma + \lambda I}_{p \times p})^{-1} \Sigma^T U^T y \\ &= \left[\begin{array}{c|c} \sigma_1 / (\sigma_1^2 + \lambda) & \\ & \ddots \\ & \sigma_p / (\sigma_p^2 + \lambda) \\ \hline & 0 \end{array} \right] \end{aligned}$$

When σ_i is large, $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx 1/\sigma_i$
(like in least squares).

When σ_i is small, $\frac{\sigma_i}{\sigma_i^2 + \lambda} \approx 0$
(so we don't make noise explode)

$$\text{Now note } (\Sigma^T \Sigma + \lambda I)^{-1} \Sigma^T = \Sigma^T (\Sigma \Sigma^T + \lambda I)^{-1}$$

i.e. $n < p$, $\Sigma_{(n \times p)} = \left[\begin{array}{c|c} \sigma_1 & \\ \sigma_2 & \\ \vdots & \\ \sigma_n & \\ \hline & 0 \end{array} \right]$
 $p-n$ cols

$$\underbrace{\Sigma^T \Sigma}_{p \times p} = \begin{bmatrix} \sigma_1^2 & & \\ & \ddots & \\ & & \sigma_n^2 \\ & & & 0 \end{bmatrix} \Rightarrow \underbrace{(\Sigma^T \Sigma + \lambda I)^{-1}}_{p \times p} \underbrace{\Sigma^T}_{p \times n} = \begin{bmatrix} \sigma_1 / (\sigma_1^2 + \lambda) & & \\ & \ddots & \\ & & \sigma_n / (\sigma_n^2 + \lambda) \\ \hline & & & 0 \end{bmatrix}$$

$$\underbrace{\Sigma \Sigma^T}_{n \times n} + \lambda I = \begin{bmatrix} \sigma_1^2 + \lambda & & \\ & \ddots & \\ & & \sigma_n^2 + \lambda \\ & & & 0 \end{bmatrix} \Rightarrow \Sigma^T (\Sigma \Sigma^T + \lambda I)^{-1} = \begin{bmatrix} \sigma_1 / (\sigma_1^2 + \lambda) & & \\ & \ddots & \\ & & \sigma_n / (\sigma_n^2 + \lambda) \\ \hline & & & 0 \end{bmatrix}$$

(similar for $p < n$)

$$\text{Then } \hat{\underline{w}} = \underbrace{V \Sigma^T U^T}_{X^T} \underbrace{U (\Sigma \Sigma^T + \lambda I)^{-1} U^T}_{\underline{\alpha}} y$$

$$\Rightarrow \hat{\underline{w}} = X^T \underline{\alpha} \quad \text{where} \quad \underline{\alpha} = (U \Sigma \Sigma^T U^T + \lambda U U^T)^{-1} y \\ = (X X^T + \lambda I)^{-1} y$$

$\Rightarrow$ that is, $\hat{\underline{w}}$ is a weighted sum of the columns of X^T
= weighted sum of the training samples

$\Rightarrow$ we can write $\hat{\underline{w}} = X^T \underline{\alpha}$ where $\underline{\alpha} \in \mathbb{R}^n$ tells us how much weight is given to each training sample

Now let's compute a ridge regression estimate in "feature space"

$$\text{Define } \Phi = \begin{bmatrix} \text{---} \phi(\underline{x}_1)^T \text{---} \\ \text{---} \phi(\underline{x}_2)^T \text{---} \\ \vdots \\ \text{---} \phi(\underline{x}_n)^T \text{---} \end{bmatrix} \in \mathbb{R}^{n \times d}$$

$$\hat{\underline{w}} = \underset{\underline{w}}{\operatorname{argmin}} \quad \|\underline{y} - \Phi \underline{w}\|_2^2 + \lambda \|\underline{w}\|_2^2$$

$$= \Phi^T \underline{\alpha} \quad \text{where } \underline{\alpha} = (\Phi \Phi^T + \lambda \mathbf{I})^{-1} \underline{y} \quad \text{from } \circledast$$

Define $K = \Phi \Phi^T$ = Kernel matrix. Then $K_{ij} = \phi(\underline{x}_i)^T \phi(\underline{x}_j)$. This is often treated like a measure of similarity between samples $\underline{x}_i$ and $\underline{x}_j$ and can be computed directly using a "Kernel function"

$k(\underline{x}_i, \underline{x}_j) := \phi(\underline{x}_i)^T \phi(\underline{x}_j)$ — bypassing any direct computation of $\phi(\underline{x}_i)$

Then $\underline{\alpha} = (K + \lambda \mathbf{I})^{-1} \underline{y}$ and for a new sample $\underline{x}$, the predicted label is

$$\hat{y} = \hat{\underline{w}}^T \phi(\underline{x}) = \underline{\alpha}^T \Phi \phi(\underline{x}) = \sum_{i=1}^n \alpha_i \phi(\underline{x}_i)^T \phi(\underline{x}) = \sum_{i=1}^n \alpha_i k(\underline{x}_i, \underline{x})$$

Kernel Ridge Regression

We seek predictions of y from $\hat{y} = \phi(\underline{x})^T \hat{\underline{w}}_k$ where

$$\hat{\underline{w}}_k = \underset{\underline{w}}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \phi(\underline{x}_i)^T \underline{w})^2 + \lambda \|\underline{w}\|_2^2$$

$$= \Phi \underline{\alpha} \quad \text{where} \quad \underline{\alpha} = (\Phi \Phi^T + \lambda I)^{-1} \underline{y}$$

We can compute such predictions efficiently via:

$$\textcircled{1} \quad \underline{\alpha} = (K + \lambda I)^{-1} \underline{y}$$

$$\textcircled{2} \quad \hat{y} = \sum_{i=1}^n \alpha_i k(\underline{x}_i, \underline{x})$$

Kernel functions

e.g. $\underline{x} \in \mathbb{R}^2$, $\underline{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, $\phi(\underline{x}) = \begin{bmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{bmatrix}$

$$K_{ij} := \phi(x_i)^T \phi(x_j) \quad \Leftrightarrow \quad K = \underbrace{\Phi \Phi^T}_{\text{matrix form}}$$

by definition

$$\begin{aligned} \Rightarrow \phi(x_i)^T \phi(x_j) &= x_{i1}^2 x_{j1}^2 + 2 x_{i1} x_{i2} x_{j1} x_{j2} + x_{i2}^2 x_{j2}^2 \quad \leftarrow \text{Computing this directly takes } O(d) \text{ time} \\ &= (x_{i1} x_{j1} + x_{i2} x_{j2})^2 \quad \text{when } \phi(x) \in \mathbb{R}^d \\ &= (\underline{x}_i^T \underline{x}_j)^2 \quad \leftarrow \text{Computing this takes } O(p) \text{ time. recall } p \ll d. \end{aligned}$$

Popular kernels

- polynomials of degree q : $K(\underline{x}_i, \underline{x}_j) = (\underline{x}_i^T \underline{x}_j)^q \quad (d = O(p^q))$
- polynomials of degree up to q : $K(\underline{x}_i, \underline{x}_j) = (\underline{x}_i^T \underline{x}_j + 1)^q \quad (d = O(p^q))$
- Gaussian Kernels: $K(\underline{x}_i, \underline{x}_j) = \exp\left(-\frac{\|\underline{x}_i - \underline{x}_j\|_2^2}{2\sigma^2}\right)$, σ^2 is a bandwidth/tuning parameter, $d = \infty$
no closed-form expression for $\phi(\underline{x})$

Note: Kernel regression gives a linear boundary in the high-dimensional feature space
 $\Leftrightarrow$ nonlinear boundary in original space.